



URL RECONNAISSANCE

Bug Bounty Hunting Guide

Crawling | Spidering | Hidden URLs | Sensitive Paths

API Discovery | robots.txt | Archives | Automation

2025 Edition

Table of Contents

1. Introduction to URL Reconnaissance

2. robots.txt & Sitemap.xml Analysis

2.1 Parsing robots.txt Directives

2.2 Sitemap.xml & Sitemap Index

2.3 .well-known/ Directory Enumeration

3. Web Archives & Historical URLs

3.1 Wayback Machine (archive.org)

3.2 Common Crawl & Other Archives

4. Web Crawling & Spidering

4.1 Katana - Fast Crawler

4.2 Gospider - JavaScript-Aware

4.3 Hakrawler - Go-based Spider

4.4 Burp Suite Spider

5. Directory & File Brute-Forcing

5.1 Ffuf - Fast Web Fuzzer

5.2 Gobuster - Directory Buster

5.3 Feroxbuster - Recursive

5.4 Dirsearch & Wfuzz

6. Hidden URL Discovery

6.1 JavaScript Route Extraction

6.2 Source Map (.map) Analysis

6.3 Hidden Parameter Discovery

7. API Endpoint Discovery

7.1 API Documentation Recon

7.2 API Path Guessing & Fuzzing

7.3 GraphQL & gRPC Discovery

8. Sensitive Files & Paths

8.1 Backup & Swap Files

8.2 Configuration & Log Files

8.3 Admin Panels & Dev Portals

9. Advanced Techniques

9.1 Wordlist Optimization

9.2 Favicon Hashing for Hidden Apps

9.3 Virtual Host Discovery

10. One-Liners & Automation

11. Tools Comparison Matrix

12. Workflow Summary

1. Introduction to URL Reconnaissance

URL reconnaissance (endpoint discovery) is the process of identifying all accessible paths, files, directories, and API endpoints on a target domain. It is one of the most critical phases in bug bounty hunting - the more URLs you discover, the larger your attack surface becomes.

Why URL Recon Matters

Hidden endpoints often expose sensitive functionality, debug interfaces, admin panels, API endpoints, and backup files that developers forgot to remove. Many critical vulnerabilities (IDOR, SSRF, LFI, RCE) are found on URLs that standard crawling misses.

Recon Category	Method	Discovery Type
Passive	Archives, cached pages, public datasets	Historical URLs, deleted pages
Passive	robots.txt, sitemap.xml, .well-known	Intentionally exposed paths
Active	Web crawling, spidering	Linked pages, JS-rendered content
Active	Directory/file brute-forcing	Hidden files, admin panels, backups
Active	Parameter discovery	Hidden query parameters, API endpoints

Pro Tip

Combine **passive** and **active** reconnaissance. Start with passive sources (archives, robots.txt) to build a seed list, then feed those URLs into active crawlers and brute-forcers for maximum coverage.

2. robots.txt & Sitemap.xml Analysis

Before launching heavy brute-force attacks, always check what the target *voluntarily* exposes. The `robots.txt` and `sitemap.xml` files are goldmines of intentionally exposed URLs.

2.1 Parsing robots.txt Directives

The `robots.txt` file tells web crawlers which paths to avoid - which ironically reveals exactly what paths exist. Additionally, it often references sitemap files.

```
# Access robots.txt
curl -s https://target.com/robots.txt

# Common patterns found:
User-agent: *
Disallow: /admin/
Disallow: /api/v1/internal/
Disallow: /debug/
Disallow: /backup/
Disallow: /test/

# Look for sitemap references
Sitemap: https://target.com/sitemap.xml
Sitemap: https://target.com/sitemap-products.xml
Sitemap: https://target.com/sitemap-users.xml.gz
```

One-Liner: Extract Disallowed Paths

Quickly extract all Disallow entries from robots.txt

```
curl -s https://target.com/robots.txt | grep -i "disallow\|allow" | awk '{print $2}' | sort -u
```

Passive

curl

⚠ Warning

Some targets include **honeypot** entries in robots.txt - paths that look interesting but trigger alerts or IP bans when accessed. Always use rate limiting and proxy rotation.

2.2 Sitemap.xml & Sitemap Index

Sitemaps list URLs the target *wants* search engines to index. A sitemap index file can reference hundreds of individual sitemap files, each containing thousands of URLs.

```
# Fetch and parse sitemap.xml
curl -s https://target.com/sitemap.xml | grep -oP '(?<=<loc>)[^<]+'

# Download sitemap index and extract all URLs
curl -s https://target.com/sitemap.xml | \
  grep -oP '(?<=<loc>)[^<]+' | \
  while read url; do curl -s "$url" | grep -oP '(?<=<loc>)[^<]+'; done

# Using Python for complex parsing
python3 -c "
import xml.etree.ElementTree as ET
import urllib.request
url = 'https://target.com/sitemap.xml'
data = urllib.request.urlopen(url).read()
root = ET.fromstring(data)
ns = {'ns': 'http://www.sitemaps.org/schemas/sitemap/0.9'}
for loc in root.iter('{http://www.sitemaps.org/schemas/sitemap/0.9}loc'):
    print(loc.text)
"
```

One-Liner: Sitemap URL Extraction with gau

```
echo "target.com" | gau --subs | grep -i sitemap
```

gau

Passive

2.3 .well-known/ Directory Enumeration

The `/.well-known/` directory is an IETF standard for exposing service metadata. Many security-critical files live here.

Path	Purpose	Bug Bounty Value
<code>/.well-known/security.txt</code>	Security contact info	Find disclosure policy, contacts
<code>/.well-known/openid-configuration</code>	OAuth/OIDC config	Auth endpoints, token URLs
<code>/.well-known/change-password</code>	Password change URL	Account takeover vectors
<code>/.well-known/assetlinks.json</code>	Android app links	App deep-linking attacks
<code>/.well-known/apple-app-site-association</code>	iOS universal links	App-to-web attack surface
<code>/.well-known/ai-plugin.json</code>	ChatGPT plugin manifest	API schema exposure
<code>/.well-known/dnt-policy.txt</code>	Do Not Track policy	Privacy compliance issues
<code>/.well-known/robots.txt</code>	Per-directory robots	Additional hidden paths
<code>/.well-known/status</code>	Health check endpoint	Info disclosure, version leaks
<code>/.well-known/nodeinfo</code>	Fediverse server info	Software version detection

```
# Quick .well-known enumeration
curl -s https://target.com/.well-known/security.txt

# Full .well-known wordlist scan (using ffuf)
ffuf -u https://target.com/.well-known/FUZZ \
  -w /usr/share/seclists/Discovery/Web-Content/well-known.txt \
  -mc 200 -t 50
```

3. Web Archives & Historical URLs

3.1 Wayback Machine (archive.org)

The Internet Archive's Wayback Machine stores historical snapshots of web pages. It often contains URLs that no longer exist on the live site - deleted features, old API versions, debug pages, and more.

```
# Basic Wayback URL extraction
curl -s "http://web.archive.org/cdx/search/cdx?url=*.target.com/*&output=text&fl=original&collapse=urlkey" | sort -u

# Using waybackurls tool
echo "target.com" | waybackurls > wayback_urls.txt

# Get only specific status codes from Wayback
curl -s "http://web.archive.org/cdx/search/cdx?url=target.com/*&output=json&fl=original,statuscode" | \
jq -r '.[1] | select(.[1] == "200") | .[0]' | sort -u

# Filter for interesting file extensions
echo "target.com" | waybackurls | grep -E '\.(js|json|xml|config|backup|sql|env|log)$'
```

One-Liner: Wayback + Ffuf Combo

Extract URLs from Wayback and feed directly into ffuf for status validation

```
echo "target.com" | waybackurls | ffuf -w - -u FUZZ -mc 200 -t 100 -o valid_wayback.txt
```

Passive

waybackurls

ffuf

3.2 Common Crawl & Other Archives

Archive	Endpoint/Tool	Notes
Common Crawl	index.commoncrawl.org	Petabyte-scale web crawl data
URLScan.io	urlscan.io/api/v1	Historical scan results with URLs
VirusTotal	virustotal.com/gui/domain/	Subdomains + URLs from scans
Spyse	spyse.com API	Cached pages and tech data
BinaryEdge	binaryedge.io API	Historical web data
OTX (AlienVault)	otx.alienvault.com	URL pulse data from community


```
# Common Crawl URL extraction using Python
python3 -c "
import requests, json
url = 'http://index.commoncrawl.org/CC-MAIN-2024-10-index'
params = {'url': '*.target.com/*', 'output': 'json', 'fl': 'url'}
r = requests.get(url, params=params)
for line in r.text.strip().split('\n'):
    if line: print(json.loads(line)['url'])
" | sort -u

# Using gau (GetAllUrls) - aggregates multiple sources
echo "target.com" | gau --o all_urls.txt --providers wayback,commoncrawl,otx,urlscan

# Using waymore for deep historical data
echo "target.com" | waymore -mode U -oU waymore_urls.txt
```

Pro Tip

Use `--dates` with waybackurls to find URLs from specific time periods. Old staging/dev URLs often persist in archives long after being removed from production.

4. Web Crawling & Spidering

Web crawlers (spiders) systematically browse a website by following links. Modern crawlers can execute JavaScript to discover dynamically rendered content and extract URLs from JS files, form actions, and AJAX endpoints.

4.1 Katana - Fast Crawler

Katana by ProjectDiscovery is a next-generation web crawler built for speed and depth. It supports headless browsing, JavaScript parsing, and automatic form filling.

```
# Basic crawl
katana -u https://target.com

# Crawl with JavaScript rendering (headless)
katana -u https://target.com -jc -jsl 1000

# Crawl multiple domains from file
katana -list domains.txt -o urls.txt

# Deep crawl with depth control
katana -u https://target.com -d 5 -jc

# Crawl and filter for specific extensions
katana -u https://target.com | grep -E '\.(php|asp|aspx|jsp|cgi)$'

# Crawl with output in JSON for parsing
katana -u https://target.com -jc -oD katana_output.json

# Crawl with custom headers and cookies
katana -u https://target.com -H "Authorization: Bearer TOKEN" -H "Cookie: session=XYZ"
```

Flag	Description
<code>-d</code>	Crawl depth (default: 3)
<code>-jc</code>	Enable JavaScript parsing
<code>-jsl</code>	JavaScript file size limit
<code>-kf</code>	Field scope (dn,rdn,fqdn)
<code>-oD</code>	Output directory
<code>-timeout</code>	Request timeout
<code>-t</code>	Number of threads

4.2 Gospider - JavaScript-Aware Spider

Gospider by @thebl4ckturtle is a fast web spider that excels at extracting URLs from JavaScript files, source maps, and robots.txt. It's particularly effective at finding API endpoints hidden in client-side code.

```
# Basic crawl
gospider -s https://target.com

# Crawl with JavaScript link extraction
gospider -S sites.txt -o output -c 10 -d 3

# Include external links and subdomains
gospider -s https://target.com --subs --other-source

# Crawl and extract API endpoints from JS
gospider -s https://target.com | grep -E '(api|graphql|rest|v[0-9])'

# Full recon: subs + external + robots + sitemap
gospider -s https://target.com --subs -o gospider_out -c 20 -d 4 --robots --sitemap
```

Katana + Gospider Combo Pipeline

```
echo "target.com" | subfinder -silent | \
  httpx -silent | \
  gospider -S - -o spider_out -c 20 --subs --other-source | \
  grep -oE 'https?://[^\s" ]+' | sort -u > all_urls.txt
```

Active

katana

gospider

subfinder

4.3 Hakrawler - Go-based Spider

Hakrawler is a fast, simple web crawler designed for quick URL discovery. It's less feature-rich than Katana but extremely fast for basic crawling tasks.

```
# Basic usage
echo "https://target.com" | hakrawler

# Crawl with subdomains included
echo "https://target.com" | hakrawler -subs

# Increase depth and include all sources
echo "https://target.com" | hakrawler -depth 3 -subs -u

# Pipe to httpx for live filtering
echo "https://target.com" | hakrawler -subs | httpx -mc 200
```

4.4 Burp Suite Spider

Burp Suite Professional includes a powerful spider/crawler that integrates with its full testing toolkit. It's particularly useful for authenticated crawling and finding content behind login walls.

```
# Burp Suite CLI crawling (using Burp's REST API)
curl -X POST 'http://localhost:1337/v0.1/scan' \
  -H 'Content-Type: application/json' \
  -d '{"urls":["https://target.com"], "scope":{"type":"Simple"}}'

# Burp Suite extensions for URL discovery:
# - Turbo Intruder: High-speed fuzzing
# - Authorize: Authorization testing (finds admin endpoints)
# - JS Link Finder: Extract URLs from JavaScript
# - Param Miner: Hidden parameter discovery
```

Crawler Selection Guide

Use **Katana** for modern JS-heavy apps, **Gospider** for deep JS analysis and source map discovery, **Hakrawler** for quick recon pipelines, and **Burp Spider** for authenticated crawling and integration with manual testing.

5. Directory & File Brute-Forcing

Brute-forcing discovers hidden directories and files by systematically guessing names using wordlists. This is an active reconnaissance technique that sends traffic directly to the target.

5.1 Ffuf - Fast Web Fuzzer

Ffuf (Fuzz Faster U Fool) is the gold standard for web fuzzing. It's extremely fast, highly configurable, and supports recursion, multiple filters, and various output formats.

```
# Basic directory brute-force
ffuf -u https://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt

# File extension brute-force
ffuf -u https://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt \
    -e .php,.bak,.txt,.xml,.config,.zip,.sql,.old

# Recursive directory brute-force
ffuf -u https://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
    -recursion -recursion-depth 3

# Use multiple wordlists simultaneously
ffuf -u https://target.com/FUZZ -w wordlist1.txt:FUZZ

# Filter by response size (exclude default 404 page)
ffuf -u https://target.com/FUZZ -w wordlist.txt -fs 2345

# Filter by status code (show only 200, 301, 302)
ffuf -u https://target.com/FUZZ -w wordlist.txt -mc 200,301,302

# Rate limiting to avoid bans
ffuf -u https://target.com/FUZZ -w wordlist.txt -t 20 -p 0.1

# Ffuf with proxy for Burp interception
ffuf -u https://target.com/FUZZ -w wordlist.txt -x http://127.0.0.1:8080

# Virtual host discovery
ffuf -u https://target.com -H "Host: FUZZ.target.com" -w subdomains.txt -fs 0
```

5.2 Gobuster - Directory Buster

Gobuster is a fast, parallel directory/file buster written in Go. It's known for its speed and simplicity.

```
# Directory brute-force
gobuster dir -u https://target.com -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

# With file extensions
gobuster dir -u https://target.com -w wordlist.txt -x php,txt,xml,bak,sql

# Include status codes and follow redirects
gobuster dir -u https://target.com -w wordlist.txt -b 404 -r

# DNS subdomain brute-force
gobuster dns -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Virtual host brute-force
gobuster vhost -u https://target.com -w subdomains.txt

# S3 bucket discovery
gobuster s3 -w bucket-names.txt

# Hide specific status codes
gobuster dir -u https://target.com -w wordlist.txt -n -e -k
```

5.3 Feroxbuster - Recursive Scanner

Feroxbuster is a recursive content discovery tool built in Rust. Its standout feature is automatic recursion - when it finds a directory, it automatically scans that directory too.

```
# Basic recursive scan
feroxbuster -u https://target.com

# Deep recursive scan with extensions
feroxbuster -u https://target.com -x php,html,js,txt,bak -d 4

# Scan with query parameter fuzzing
feroxbuster -u https://target.com?FUZZ=test -w params.txt

# Extract links from responses and add to scan queue
feroxbuster -u https://target.com --extract-links

# Filter by response size
feroxbuster -u https://target.com -S 1000

# Rate limiting and timeout configuration
feroxbuster -u https://target.com -t 50 --time-limit 30m

# Scan through SOCKS5 proxy
feroxbuster -u https://target.com --proxy socks5://127.0.0.1:9050
```

5.4 Dirsearch & Wfuzz

```
# Dirsearch - Python-based directory scanner
python3 dirsearch.py -u https://target.com -e php,html,js,xml,bak

# Dirsearch with recursive scan
python3 dirsearch.py -u https://target.com -e * -r -R 3

# Wfuzz - general purpose web fuzzer
wfuzz -c -z file,/usr/share/wordlists/dirb/common.txt --hc 404 https://target.com/FUZZ

# Wfuzz for hidden virtual hosts
wfuzz -c -z file,subdomains.txt -H "Host: FUZZ.target.com" --hc 404 https://target.com

# Wfuzz for parameter discovery
wfuzz -c -z file,params.txt "https://target.com/page.php?FUZZ=value"
```

Rate Limiting is Critical

Directory brute-forcing generates significant traffic. Always use `-t 20` or lower threads, add delays with `-p`, and use proxies. Many bug bounty programs explicitly ban aggressive scanning without rate limiting.

6. Hidden URL Discovery

Modern web applications load significant functionality through JavaScript. Hidden routes, API endpoints, and debug interfaces often exist only in client-side code.

6.1 JavaScript Route Extraction

Single Page Applications (SPAs) define routes in JavaScript. Extracting these reveals endpoints that server-side crawling completely misses.

```
# Manual extraction using grep/ripgrep
curl -s https://target.com/app.js | grep -oE '"/[a-zA-Z0-9_/-]+' | sort -u

# Using ripgrep for better pattern matching
curl -s https://target.com/app.js | rg -o '"/api/[a-zA-Z0-9_/-]+' | sort -u

# Extract fetch/XHR endpoints
curl -s https://target.com/app.js | grep -oE 'fetch\(["\x27][^\x27]+\["\x27]' | sort -u

# Extract axios/baseURL configurations
curl -s https://target.com/app.js | grep -oE 'baseURL["\x27]*\s*:\s*["\x27][^\x27]+'

# Automated with LinkFinder
python3 linkfinder.py -i https://target.com/app.js -o cli

# Using getJS for bulk JS analysis
echo "target.com" | getJS --complete | while read js; do
  echo "=== $js ==="
  python3 linkfinder.py -i "$js" -o cli
done

# Using mantra for endpoint extraction
echo "https://target.com" | mantra -t 20 | tee js_endpoints.txt
```

Regex Pattern	What It Finds
<code>/api/[vV]?[0-9]+/["'']\s+</code>	API endpoint paths
<code>fetch\(["'']["'']+\["'']</code>	Fetch API calls
<code>axios\.[getpost]+\(["'']["'']+</code>	Axios HTTP requests
<code>path\s*:\s*["'']["'']+\["'']</code>	React/Vue router paths
<code>url\s*:\s*["'']["'']+\["'']</code>	Generic URL configs
<code>href\s*=\s*["'']["'']+\["'']</code>	Link href values
<code>/[a-zA-Z0-9_/-]+/[a-zA-Z0-9_/-]+\.[a-z]{2,4}</code>	File paths in JS

6.2 Source Map (.map) Analysis

Source maps (`.js.map` files) map minified code back to original source. If exposed, they reveal the entire unminified codebase - complete with comments, original filenames, and

hidden endpoints.

```
# Check if source maps are exposed
curl -s https://target.com/static/js/app.js | grep "sourceMappingURL"

# If found, download and analyze the map
curl -s https://target.com/static/js/app.js.map | python3 -m json.tool | head -50

# Extract original source filenames from map
curl -s https://target.com/app.js.map | jq -r '.sources[]'

# Full source map decompilation with sourcemap-extractor
npm install -g sourcemap-extractor
sourcemap-extractor -u https://target.com/app.js.map -o source_output/

# Using Python for automated source map analysis
python3 -c "
import json, requests, re
url = 'https://target.com/app.js.map'
data = requests.get(url).json()
# Extract all strings that look like API paths
sources = ' '.join(data.get('sources', []))
paths = re.findall(r'[a-zA-Z0-9_/-]+', sources)
for p in sorted(set(paths)):
    print(p)
"
```

Source Map Bug Bounty Impact

Exposed source maps are a valid bug bounty finding (information disclosure). They reveal: original variable names, internal IP addresses, API keys in comments, hidden admin routes, third-party integrations, and the complete application architecture. Always report if found.

6.3 Hidden Parameter Discovery

Web applications often support parameters that aren't documented or visible in the UI. Discovering these can lead to IDOR, privilege escalation, and access to hidden features.

```
# Arjun - Hidden parameter discovery
arjun -u https://target.com/api/profile

# Arjun with custom wordlist
arjun -u https://target.com/api/search -w params.txt -m GET,POST

# Arjun with JSON body support
arjun -u https://target.com/api/data -m POST --json -w params.txt

# Param Miner (Burp Suite extension)
# 1. Right-click request -> Extensions -> Param Miner -> Guess params
# 2. Supports headers, cookies, body params
# 3. Uses binary search for efficiency

# x8 - Hidden parameter discovery written in Rust
x8 -u "https://target.com/page.php" -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt

# Manual parameter guessing with ffuf
ffuf -u "https://target.com/page.php?FUZZ=test" \
    -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
    -fs 0 -mc 200
```

Tool	Language	Key Feature
Arjun	Python	Multi-threaded, GET/POST/JSON support
Param Miner	Java (Burp)	Binary search, header param guessing
x8	Rust	Extremely fast, multiple test values
ffuf	Go	Flexible, can fuzz any parameter position

7. API Endpoint Discovery

APIs are the backbone of modern applications. Discovering undocumented API endpoints is one of the highest-value activities in bug bounty hunting.

7.1 API Documentation Recon

```
# Common API documentation paths to check
curl -s https://target.com/api
curl -s https://target.com/api/docs
curl -s https://target.com/swagger.json
curl -s https://target.com/swagger-ui.html
curl -s https://target.com/v2/api-docs
curl -s https://target.com/openapi.json
curl -s https://target.com/api/v1/openapi.yaml
curl -s https://target.com/graphql

# If swagger is found, extract all endpoints
curl -s https://target.com/swagger.json | jq -r '.paths | keys[]'

# Extract and test all API endpoints from swagger
curl -s https://target.com/swagger.json | \
  jq -r '.paths | to_entries[] | .key as $p | .value | keys[] as $m | "\($m | ascii_
  upcase) \($p)'"

# Postman collection discovery
curl -s https://target.com/postman/collection.json | jq '.item[].name'
```

Documentation Path	Framework	Extraction Command
/swagger.json	Swagger 2.0	jq '.paths keys[]'
/openapi.json	OpenAPI 3.0	jq '.paths keys[]'
/v2/api-docs	Springfox	jq '.paths keys[]'
/api-docs	Various	jq '.paths keys[]'
/graphql	GraphQL	Introspection query
/playground	Apollo	Interactive schema browser
/altair	GraphQL	Altair client UI

7.2 API Path Guessing & Fuzzing

```
# API version enumeration
ffuf -u https://target.com/api/FUZZ/users -w versions.txt -mc 200
# versions.txt contents: v1, v2, v3, 1, 2, 3, internal, private, public, beta, alpha

# Common API path patterns
ffuf -u https://target.com/FUZZ -w api_paths.txt -mc 200
# api_paths.txt: api, API, rest, graphql, grpc, ws, websocket, internal, svc

# Fuzz API resource names
ffuf -u https://target.com/api/v1/FUZZ -w resources.txt -mc 200
# resources.txt: users, accounts, orders, payments, admin, config, settings, uploads, files, auth, login

# Fuzz for API parameter pollution
ffuf -u "https://target.com/api/users?id=1&id=2&id=FUZZ" -w ids.txt

# Test for mass assignment by adding extra fields
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"test","email":"test@test.com","role":"admin","is_admin":true}'
```

7.3 GraphQL & gRPC Discovery

```
# GraphQL introspection query (if enabled)
curl -X POST https://target.com/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { types { name fields { name type { name } } } } }" }' | \
jq

# Using GraphQL introspection to find queries/mutations
curl -X POST https://target.com/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { fields { name args { name } } } mutationType { fields { name } } } }" }' | jq

# GraphQL endpoint discovery with common paths
echo -e "/graphql\n/api/graphql\n/graphql/v1\n/query\n/api/query" | \
ffuf -u https://target.com/FUZZ -w - -X POST \
  -H "Content-Type: application/json" \
  -d '{"query":"{__typename}" }' -mr "__typename"

# gRPC reflection (if enabled)
grpcurl -plaintext target.com:443 list
grpcurl -plaintext target.com:443 list mypackage.MyService

# gRPC Web endpoint detection
curl -s https://target.com/grpc -H "Content-Type: application/grpc-web+proto"
```

API Testing Tip

Always test for **HTTP method switching**. An endpoint that only validates **GET** **/api/users/123** might not properly authorize **DELETE** **/api/users/123** or **PUT** **/api/users/123**. Use ffuf with **-X POST,PUT,DELETE,PATCH** to test all methods on discovered endpoints.

8. Sensitive Files & Paths

8.1 Backup & Swap Files

Developers frequently leave backup copies of critical files. These can contain source code, database credentials, and configuration secrets.

```
# Common backup file patterns to fuzz
ffuf -u https://target.com/FUZZ -w backup_extensions.txt -mc 200
# backup_extensions.txt:
# index.php.bak
# index.php~
# index.php.old
# index.php.swp
# index.php.save
# .index.php.swp
# .env.backup
# config.php.bak
# database.sql.gz
# backup.zip
# site.tar.gz
# dump.sql
# www.zip
# source.zip
# backup.tar.gz

# Automated backup discovery using bfac (Backup File Artifacts Checker)
bfac --url https://target.com

# Test for .git folder exposure
curl -s https://target.com/.git/config
curl -s https://target.com/.git/HEAD

# If .git is exposed, dump the entire repository
git-dumper https://target.com/.git/ ./target-source/

# Test for .svn and .hg exposure
curl -s https://target.com/.svn/entries
curl -s https://target.com/.hg/store/00manifest.i

# Test for .DS_Store files (macOS - reveals directory listing)
curl -s https://target.com/.DS_Store | strings | head -50
```

File Pattern	Description	Risk Level
<code>*.bak, *.backup</code>	Generic backup files	High
<code>*~, *.swp, *.swo</code>	Vim swap files	High
<code>.env, .env.local</code>	Environment variables	Critical
<code>*.sql, *.sql.gz</code>	Database dumps	Critical
<code>*.zip, *.tar.gz, *.rar</code>	Archive files	High
<code>.git/, .svn/, .hg/</code>	Version control repos	Critical
<code>.DS_Store</code>	macOS directory metadata	Medium
<code>*.old, *.orig</code>	Old file versions	Medium

8.2 Configuration & Log Files

```
# Configuration file discovery
ffuf -u https://target.com/FUZZ -w config_files.txt -mc 200
# config_files.txt:
# .env
# .env.local
# .env.production
# config.json
# config.yaml
# config.yml
# application.properties
# application.yml
# web.config
# appsettings.json
# settings.py
# database.yml
# docker-compose.yml
# package.json
# composer.json
# requirements.txt
# Gemfile

# Log file discovery
curl -s https://target.com/logs/
curl -s https://target.com/log/
curl -s https://target.com/access.log
curl -s https://target.com/error.log
curl -s https://target.com/debug.log
curl -s https://target.com/application.log
curl -s https://target.com/storage/logs/laravel.log
```

Responsible Disclosure

If you find exposed .env files containing database credentials, API keys, or passwords, **stop immediately**. Do not download or access the data beyond confirming its existence. Report it as a Critical/P1 finding immediately.

8.3 Admin Panels & Dev Portals

```
# Common admin path brute-forcing
ffuf -u https://target.com/FUZZ -w admin_paths.txt -mc 200,301
# admin_paths.txt:
# admin
# administrator
# admin.php
# admin/login
# adminpanel
# panel
# dashboard
# manage
# management
# wp-admin
# phpmyadmin
# phpMyAdmin
# cpanel
# whm
# webmail
# roundcube
# api/admin
# internal
# staff
# employee
# backend
# console
# dev
# development
# staging
# test
# uat
# qa
# debug
# phpinfo.php
# server-status
# server-info
# actuator
# actuator/env
# actuator/health
# actuator/httptrace
# swagger-ui.html

# Check for default admin panels
curl -s https://target.com/phpmyadmin/
curl -s https://target.com/cpanel/
curl -s https://target.com/wp-admin/
curl -s https://target.com/adminer.php
```

9. Advanced Techniques

9.1 Wordlist Optimization

The quality of your wordlist determines brute-force effectiveness. Generic wordlists miss application-specific paths. Build custom wordlists from the target itself.

```
# Build custom wordlist from target content
curl -s https://target.com | tr '[:space:]' '\n' | grep -oE '[a-zA-Z0-9_-]{3,}' | sort -u > custom_words.txt

# Extract words from all discovered JS files
cat urls.txt | grep '\.js$' | while read url; do
    curl -s "$url" | tr '[:space:]' '\n' | grep -oE '[a-zA-Z0-9_-]{4,}'
done | sort -u > js_words.txt

# Combine multiple wordlists and deduplicate
cat /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
    /usr/share/seclists/Discovery/Web-Content/raft-medium-files.txt \
    custom_words.txt | sort -u > combined_wordlist.txt

# Use CeWL (Custom Word List generator)
cewl https://target.com -d 3 -m 4 -w cewl_words.txt

# Generate permutations with altdns
cat base_words.txt | altdns -w permutations.txt -o expanded.txt

# Filter wordlist by response analysis
ffuf -u https://target.com/FUZZ -w big_wordlist.txt -ac -o fuzz_results.json
```

9.2 Favicon Hashing for Hidden Apps

Every web application has a favicon with a unique hash. By calculating the MurmurHash of a favicon and searching it in Shodan, you can discover other instances of the same application running on different ports or subdomains.

```
# Get favicon and calculate hash
curl -s https://target.com/favicon.ico | python3 -c "
import mmh3, sys, base64
content = sys.stdin.buffer.read()
hash_val = mmh3.hash(content)
print(f'MurmurHash: {hash_val}')
print(f'Base64: {base64.b64encode(content).decode()}')
"

# Search Shodan by favicon hash
shodan search "http.favicon.hash:-123456789"

# Using httpx for favicon hash extraction
echo "target.com" | httpx -favicon -json | jq -r '.favicon_mmh3'

# Search multiple targets for interesting favicons
subfinder -d target.com | httpx -favicon -json | jq -r 'select(.favicon_mmh3 != "")
| [.url, .favicon_mmh3] | @tsv'
```

9.3 Virtual Host Discovery

Sometimes different applications or versions are served based on the Host header. Virtual host discovery finds these hidden applications.

```
# Virtual host brute-forcing with ffuf
ffuf -u https://target.com -H "Host: FUZZ.target.com" \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-fs 0 -mc 200,301,302

# Compare baseline response size first
ffuf -u https://target.com -H "Host: random.target.com" -fs 0

# Virtual host discovery with Gobuster
gobuster vhost -u https://target.com -w subdomains.txt

# Test for Host header injection / cache poisoning
curl -s https://target.com -H "Host: evil.com" -I
```

10. One-Liners & Automation

Complete Recon Pipeline (Passive to Active)

```
# Full URL recon pipeline
subfinder -d target.com -silent | \
  httpx -silent -mc 200,301,302 | \
  tee live_hosts.txt | \
  katana -list - -jc -d 5 | \
  tee katana_urls.txt | \
  gau --subs | \
  sort -u | \
  tee all_urls.txt | \
  httpx -silent -mc 200 | \
  tee valid_urls.txt
```

PassiveActive

JavaScript Endpoint Extraction Pipeline

```
# Extract all JS URLs, then extract endpoints from each
cat urls.txt | subjs | while read js; do
  echo "--- $js ---"
  python3 linkfinder.py -i "$js" -o cli 2>/dev/null
done | grep -oE '(/[a-zA-Z0-9_/.-]+|https?://[^( ]+)' | sort -u
```

subjslinkfinder

Sensitive File Discovery

```
# Find exposed .env, config, and backup files
cat live_hosts.txt | while read host; do
  ffuf -u "${host}/FUZZ" -w sensitive_files.txt -t 30 -mc 200 -fs 0
done

# sensitive_files.txt:
# .env
# .env.local
# config.json
# web.config
# backup.sql
# database.sql
# dump.sql
# phpinfo.php
# .htaccess
# .htpasswd
```

Active

ffuf

API Endpoint Enumeration

```
# Find API docs, then enumerate all endpoints
cat live_hosts.txt | while read host; do
  for path in /api /swagger.json /openapi.json /v2/api-docs /graphql; do
    status=$(curl -s -o /dev/null -w "%{http_code}" "${host}${path}")
    echo "${host}${path} -> ${status}"
  done
done

# If swagger found, extract and test all endpoints
curl -s https://target.com/swagger.json | \
  jq -r '.paths | to_entries[] | .key as $p | .value | keys[] as $m | "\($m) \($p)"' | \
  while read method path; do
    curl -s -X "$method" "https://target.com${path}" -w " %{http_code}\n"
  done
```

Active

curl

jq

Automated Parameter Discovery

```
# Run Arjun on all discovered GET endpoints
cat valid_get_urls.txt | while read url; do
    arjun -u "$url" -oT arjun_results.txt -t 10
done

# Run x8 on all POST endpoints
cat valid_post_urls.txt | while read url; do
    x8 -u "$url" -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-name
s.txt \
    -X POST --reflections
done
```

Active

arjun

x8

Wayback + Crawler + Brute-force Mega Pipeline

```
# Ultimate URL discovery pipeline
echo "target.com" | \
    subfinder -silent | \
    httpx -silent | \
    tee hosts.txt | \
    while read host; do
        # Passive
        echo "$host" | gau --o "wb_${host}.txt"
        # Active crawl
        katana -u "$host" -jc -d 3 -o "katana_${host}.txt"
        # Brute-force directories
        ffuf -u "${host}/FUZZ" -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
            -t 30 -mc 200,301,302 -ac -o "ffuf_${host}.json"
    done

# Merge all results
cat wb_*.txt katana_*.txt 2>/dev/null | sort -u > all_discovered_urls.txt
```

Passive

Active

11. Tools Comparison Matrix

Tool	Type	Speed	JS Support	Recursion	Auth	Best For
Katana	Crawler	Fast	Full (headless)	Yes	Headers/Cookie	Modern SPA crawling
Gospider	Spider	Fast	Link extraction	Yes	Basic	JS file analysis
Hakrawler	Crawler	Very Fast	No	Yes	No	Quick recon pipelines
Burp Spider	Crawler	Medium	Full (headless)	Yes	Full session	Authenticated crawling
Ffuf	Fuzzer	Very Fast	N/A	Manual	Full	Directory/file brute-force
Gobuster	Fuzzer	Fast	N/A	No	Basic	Quick directory scans
Feroxbuster	Fuzzer	Fast	Link extraction	Auto	Basic	Recursive deep scanning
Dirsearch	Fuzzer	Medium	No	Yes	Basic	Python-based flexibility
Arjun	Param finder	Medium	N/A	N/A	Basic	Hidden parameter discovery
Param Miner	Param finder	Fast	N/A	N/A	Burp session	Burp Suite integration
x8	Param finder	Very Fast	N/A	N/A	Basic	High-speed param fuzzing
waybackurls	Archive	Fast	N/A	N/A	No	Historical URL discovery
gau	Archive	Fast	N/A	N/A	No	Multi-source URL aggregation

LinkFinder	JS Analyzer	Medium	Analysis	N/A	No	JS endpoint extraction
------------	-------------	--------	----------	-----	----	------------------------

12. Workflow Summary

- 1 **Passive Recon** - Start with robots.txt, sitemap.xml, and .well-known/ directory. No traffic hits the target. Extract all URLs from these sources to build your seed list.
- 2 **Archive Mining** - Query Wayback Machine, Common Crawl, and URLScan.io for historical URLs. These often reveal deleted endpoints, old API versions, and debug pages.
- 3 **Web Crawling** - Feed discovered URLs into Katana (JS-enabled) and Gospider. Let crawlers follow links and extract JavaScript-based endpoints. This finds dynamically rendered content.
- 4 **JavaScript Analysis** - Extract all JS files found during crawling. Run LinkFinder and manual regex analysis to discover API endpoints, router definitions, and hidden routes. Check for exposed .map files.
- 5 **Directory Brute-Forcing** - Use Ffuf or Feroxbuster with custom wordlists built from the target's own content. Test for backup files, config files, and admin panels. Always rate-limit.
- 6 **API Discovery** - Search for swagger/openapi docs, fuzz API path patterns, test GraphQL endpoints, and attempt introspection queries. Test all HTTP methods on each endpoint.
- 7 **Parameter Discovery** - Run Arjun or Param Miner on all discovered endpoints. Hidden parameters often lead to IDOR, access control bypasses, and hidden features.
- 8 **Sensitive File Hunt** - Check for .env, backup files, version control repos (.git), log files, and admin panels. Use bfac and git-dumper for automated detection.
- 9 **Consolidation** - Merge all discovered URLs, deduplicate, and filter by status code. Feed the final list into vulnerability scanners or manual testing workflows.

Remember

URL reconnaissance is an iterative process. Each discovered URL can lead to new JavaScript files, which contain more endpoints, which reveal new parameters, which expose hidden functionality. The deeper you go, the more bugs you find. Always document your findings and respect the program's scope and rate limits.